

SPACE INVADERS



A 2D ARCADE GAME IN
PYTHON USING PYGAME

OUR TEAM

1. LAKSHMI MAHESWARI -AP23110011590

2. HIMAKSHI- AP23110011596

3. ADI LAKSHMI - AP23110011599

4. GNANASREE- AP23110011650

INTRODUCTION/PROBLEM STATEMENT

- This project involves recreating the classic Space Invaders arcade game using Python and Pygame.
- The objective is to build a 2D shooter where the player controls a spaceship to destroy alien enemies.
- The project addresses how to integrate graphics, sound, and real-time interactivity in a Python environment.
- Relevance: Demonstrates practical implementation of game development principles using beginner-friendly tools.

OBJECTIVES / HYPOTHESES

Main objective:

To develop a 2D game that features real-time enemy movement, collision detection, sound effects, score tracking, and increasing difficulty.

Built on concepts from:

- 2D game development
- Object-oriented programming
- Collision detection in computer graphics
- Game loop and state management

METHODOLOGY

Pygame was used to render graphics and handle sound

Game development steps:

- Set up window and game loop
- Add player sprite and movement logic
- Add enemies with movement
- Add bullet and shooting mechanism
- Detect collision and update score
- Add game over and high score logic

CODE STRUCTURE OVERVIEW

Main Files

File 1: main.py – Basic version of Space Invaders

File 2: levelup.py – Enhanced version with levels, high score, and sound menu

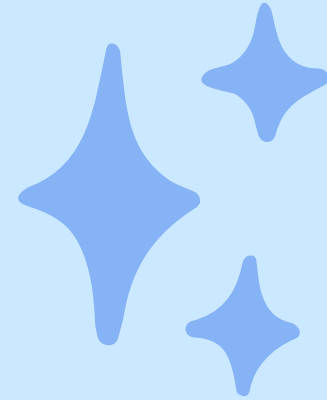
Key functions

```
def player(x, y):  
    screen.blit(playerImg, (x, y))  
  
def enemy(x, y, i):  
    screen.blit(enemyImg[i], (x, y))  
  
def fire_bullet(x, y):  
    global bullet_state  
    bullet_state = "fire"  
    screen.blit(bulletImg, (x + 16, y + 10))  
  
def isCollision(enemyX, enemyY, bulletX, bulletY):  
    distance = math.sqrt(math.pow(enemyX - bulletX, 2) + (math.pow(enemyY - bulletY, 2)))  
    if distance < 27:  
        return True  
    else:  
        return False
```

IMPLEMENTATION/ WORKFLOW

Workflow:

1. Initialize Pygame
2. Load assets
3. Main game loop
4. Handle inputs
5. Update positions and states
6. Detect collisions
7. Render updated screen



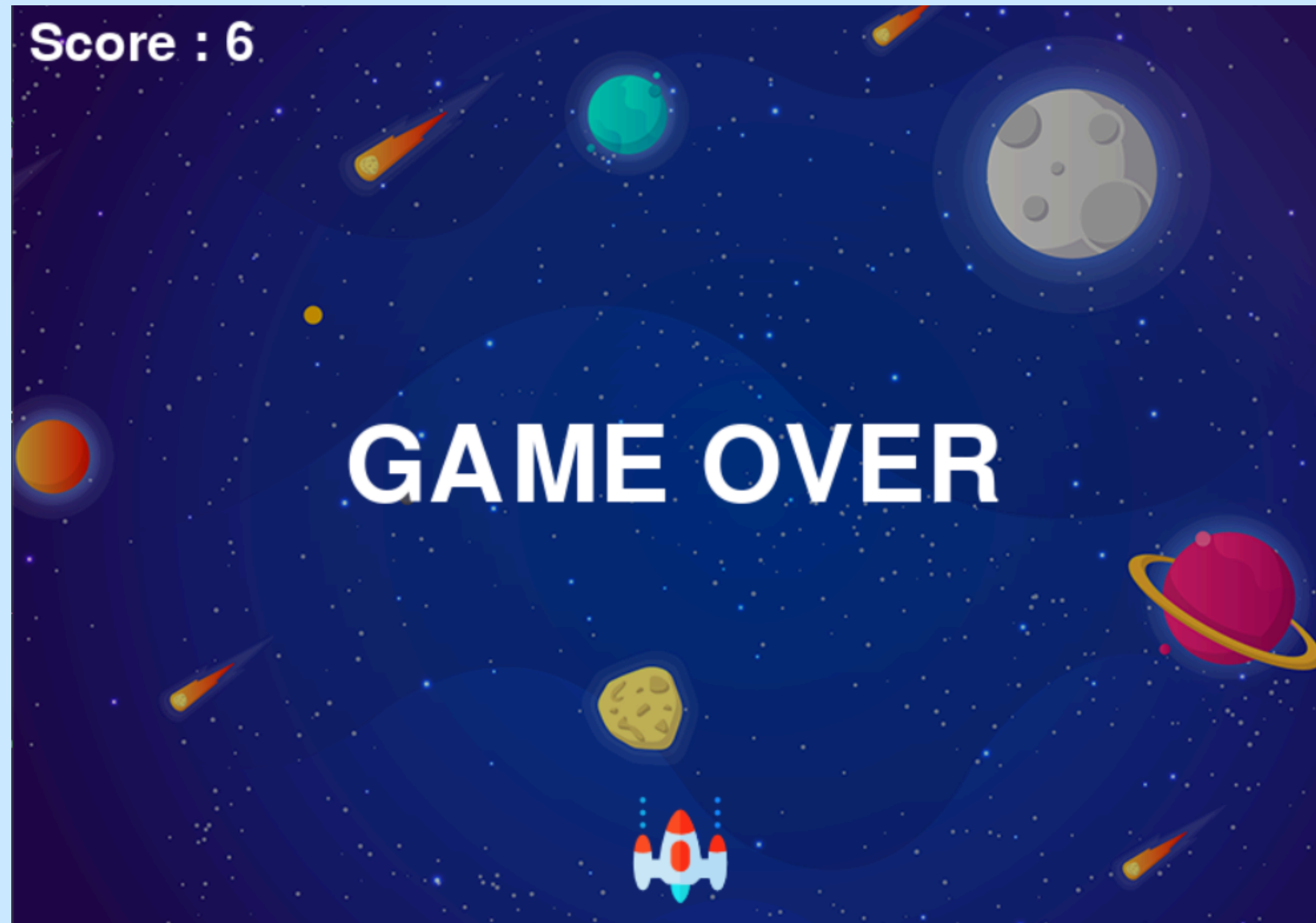
GAME OVER CONDITION

Player loses if enemies cross a certain line.

```
# Enemy Movement
for i in range(num_of_enemies):

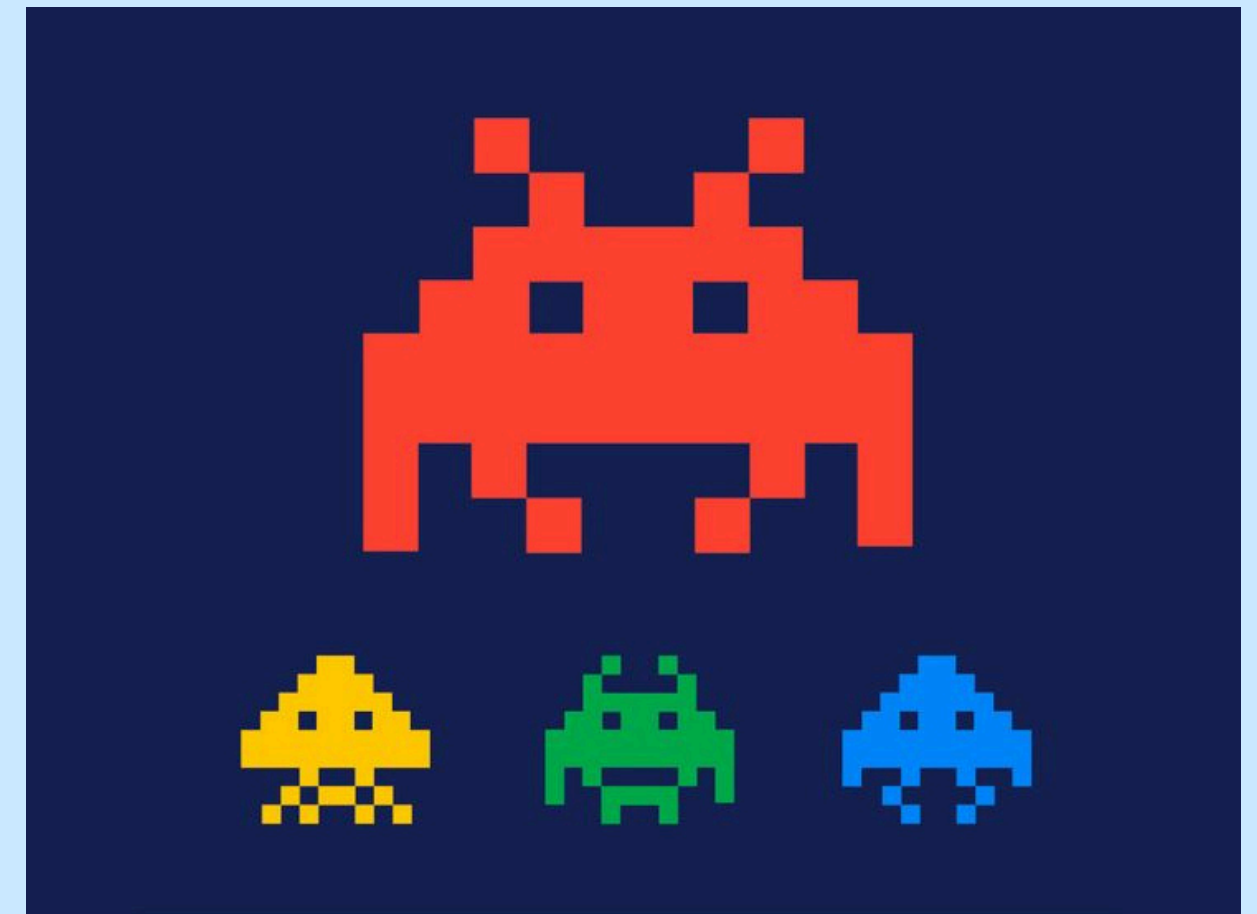
    # Game Over
    if enemyY[i] > 440:
        for j in range(num_of_enemies):
            enemyY[j] = 2000
        game_over_text()
        break

    enemyX[i] += enemyX_change[i]
    if enemyX[i] <= 0:
        enemyX_change[i] = 4
        enemyY[i] += enemyY_change[i]
    elif enemyX[i] >= 736:
        enemyX_change[i] = -4
        enemyY[i] += enemyY_change[i]
```



RESULTS

- Successfully developed a playable 2D space shooting game.
- Core Features:
 - Shooting mechanics and collision detection
 - Sound integration (background music, laser, explosion)
 - Game Over and restart functionality
 - Level progression based on score
 - High score saving and tracking
- Visual feedback through animations and a responsive interface.



CONCLUSION

- Space Invaders using Python and Pygame proves that game logic, interactivity, and performance can be achieved even with a simple tech stack.
- It is both a learning tool and a creative application of coding principles.
- Helped strengthen my knowledge of:
 - Event handling
 - Game loop logic
 - Collision mathematics
 - Real-time system response

**THANK
YOU**